

# Implementación definitiva tesis

1. Descargar OpenFoam desde:

```
git clone --branch OpenFOAM-v2406 https://develop.openfoam.com/Development/openfoam.git ~/OpenFOAM-2406
```

cargar el entorno

```
source ~/OpenFOAM-2406/etc/bashrc
```

compilar con wmake

2. Hacer una copia del solver pimpleFoam para trabajar desde ahí y no perder archivos importantes

Cambiar nombre del ejecutable en Make/files

```
desnitrificationFoam.C  
EXE = $(FOAM_USER_APPBIN)/desnitrificationFoam
```

Copiar librería que contiene timestepManager

La librería `NumericalMethods`, que contiene el componente `timestepManager`, fue extraída directamente desde el solver oficial `biofilmFoam`, específicamente desde la ruta: `~/biofilmFoam/libraries/numericalMethods/`

## Copiar la librería al entorno de trabajo

```
mkdir -p ~/desnitrificationFoam/libraries/numericalMethods  
cp -r ~/biofilmFoam/libraries/numericalMethods/* ~/desnitrificationFoam/libraries/numericalMethods/
```

```
cd ~/desnitrificationFoam/libraries/numericalMethods
wmake
```

Modificar `Make/options` del solver

```
cat > ~/hito2/hito2/Make/options << 'EOF'
EXE_INC = \
    -I$(LIB_SRC)/finiteVolume \
    -I$(LIB_SRC)/finiteVolume/InInclude \
    -I$(LIB_SRC)/meshTools/InInclude \
    -I$(LIB_SRC)/sampling/InInclude \
    -I$(LIB_SRC)/TurbulenceModels/turbulenceModels/InInclude \
    -I$(LIB_SRC)/TurbulenceModels/incompressible/InInclude \
    -I$(LIB_SRC)/transportModels \
    -I$(LIB_SRC)/transportModels/incompressible/singlePhaseTransportModel \
    -I$(LIB_SRC)/dynamicMesh/InInclude \
    -I$(LIB_SRC)/dynamicFvMesh/InInclude \
    -I../libraries/numericalMethods/InInclude #agregar la ruta de la libreria c
ompilada

EXE_LIBS = \
    -lfiniteVolume \
    -lfvOptions \
    -lmeshTools \
    -lsampling \
    -lturbulenceModels \
    -lincompressibleTurbulenceModels \
    -lincompressibleTransportModels \
    -ldynamicMesh \
    -ldynamicFvMesh \
    -ltopoChangerFvMesh \
    -latmosphericModels \
    -L$(FOAM_USER_LIBBIN) \
    -lNumericalMethods
```

```
EOF
```

```
wclean  
wmake
```

Crear archivos .H necesarios para la simulacion de la biopelícula , todos estos archivos son piezas modulares que se incluyen en el solver principal (.C) para realizar tareas específicas.

## Archivo Principal y Estructura

▼ **desnitrificationFoam.C** : Es el archivo maestro. Contiene el bucle de tiempo principal, coordina el algoritmo PIMPLE, incluye a todos los demás archivos **.H** en el orden correcto y gestiona la escritura de datos.

```
/*-----  
*\n  
===== |  
\\  / F ield      | OpenFOAM: The Open Source CFD Toolbox  
\\  / O peration  |  
\\  / A nd         | www.openfoam.com  
\\  / M anipulation |  
-----  
  
--  
  
Copyright (C) 2011-2017 OpenFOAM Foundation  
Copyright (C) 2019 OpenCFD Ltd.  
-----  
  
--
```

## License

This file is part of OpenFOAM.

OpenFOAM is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

OpenFOAM is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or

FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License

for more details.

You should have received a copy of the GNU General Public License along with OpenFOAM. If not, see <<http://www.gnu.org/licenses/>>.

## Application

biofilmPimpleFoam

```
\*-----  
*/
```

```
#include "fvCFD.H"  
#include "dynamicFvMesh.H"  
#include "singlePhaseTransportModel.H"  
#include "turbulentTransportModel.H"  
#include "pimpleControl.H"  
// #include "CorrectPhi.H"  
#include "cfdTools/general/CorrectPhi/CorrectPhi.H"  
#include "fvOptions.H"  
#include "timestepManager.H"
```

```
// ***** //
```

```

int main(int argc, char *argv[])
{
    #include "postProcess.H"

    #include "addCheckCaseOptions.H"
    #include "setRootCaseLists.H"
    #include "createTime.H"
    #include "createDynamicFvMesh.H"
    #include "initContinuityErrs.H"
    #include "createDyMControls.H"

    #include "createFieldsBiofilm.H"
    #include "createFieldsNOx.H"
    #include "createFields.H"

    #include "createUflfPresent.H"
    #include "CourantNo.H"

    #include "readTimeControls.H"
    // Permite activar/desactivar la ecuación de C desde system/controlDict
    const Switch useC = runTime.controlDict().lookupOrDefault<Switch>("useC", true);
    // Permite activar/desactivar la convección de NO3, NO2, Csb y SO4
    const Switch advectSolutes = runTime.controlDict().lookupOrDefault<Switch>("advectSolutes", true);

    #include "setInitialDeltaT.H"

    turbulence->validate();

    // * * * * * //

    Info << "\nStarting time loop\n" << endl;

    Info<< nl
        << "ADVECTIVE SOLUTE TRANSPORT: " << (advectSolutes ? "ON" :

```

```

"OFF") << nl
    << "useC: " << (useC ? "ON" : "OFF") << nl << endl;

while (runTime.run())
{

#include "CourantNo.H"
#include "setDeltaT.H"

++runTime;

Info << "Time = " << runTime.timeName() << nl << endl;

#include "pimple.H"
    if (useC)
    {
        #include "CEqn.H"
    }

#include "updateDiffusion.H"
    #include "MHidEqn.H"
#include "NOxEqn.H"
#include "MEqn.H"
    #include "updateInterfaceMS.H"
#include "updateBiofilmPhase.H"

    if (useC) dtManagerC.updateDerivatives();
dtManagerM.updateDerivatives();
    dtManagerNO3.updateDerivatives();
    dtManagerNO2.updateDerivatives();

    if (useC)
    {
        Info << "C in [" << gMin(C) << ", " << gMax(C) << "]" << endl;
    }
}

```

```

Info << "M in [" << gMin(M) << ", " << gMax(M) << "]" << endl;
Info << "NO3 in [" << gMin(NO3) << ", " << gMax(NO3) << "]" << endl;
l;
Info << "NO2 in [" << gMin(NO2) << ", " << gMax(NO2) << "]" << endl;
l;
Info << "Csb in [" << gMin(Csb) << ", " << gMax(Csb) << "]" << endl;
Info << "SO4 in [" << gMin(SO4) << ", " << gMax(SO4) << "]" << endl;
l;
Info << "Mhid in [" << gMin(Mhid) << ", " << gMax(Mhid) << "]" << endl;
Info << "S0 in [" << gMin(S0) << ", " << gMax(S0) << "]" << endl;
runTime.write();

runTime.printExecutionTime(Info);
}

Info << "End\n" << endl;

return 0;
}

// *****
**** //

// === MHidEqn.H: hidrólisis S0→Sb + difusión de biomasa hidrolítica ===
===
// Evitar divisiones por cero (mismas dimensiones que S0)
const dimensionedScalar tinyS0("tinyS0", S0.dimensions(), SMALL);

//  $\mu_{hid} = k1 * S0 / (K1\_over\_aStar + S0)$ 
volScalarField mu_hid
(
    IOobject("mu_hid", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::NO_WRITE),
    k1 * S0 / (K1_over_aStar + S0 + tinyS0) // [1/s]

```

```

);

// Tasa de hidrólisis volumétrica (kg S0/m3/s), se mantiene igual
volScalarField R_hyd
(
    IOobject("R_hyd", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::AUTO_WRITE),
    mu_hid * Mhid
);

// PDE con difusión:
//  $\frac{dM_{hid}}{dt} - \nabla \cdot (D \nabla M_{hid}) = (K_0 \mu_{hid} - k_d_{hid}) * M_{hid}$ 
fvScalarMatrix MHidEqn
(
    fvm::ddt(Mhid)
    - fvm::laplacian(D, Mhid)
    ==
    (K0*mu_hid - kd_hid) * Mhid
);

MHidEqn.solve();
// =====
=====

```

## Archivos de inicialización de campos:

- ▼ **createFields.H** : Inicializa las variables hidrodinámicas básicas de OpenFOAM . También lee inicialmente la porosity y K (permeabilidad) y crea el modelo de turbulencia/laminar.



```

Info << "Reading field p\n" << endl;
volScalarField p
(
    IOobject
    (
        "p",
        runTime.timeName(),
        mesh,
        IOobject::MUST_READ,
        IOobject::AUTO_WRITE
    ),
    mesh
);

Info << "Reading field U\n" << endl;
volVectorField U
(
    IOobject
    (
        "U",
        runTime.timeName(),
        mesh,
        IOobject::MUST_READ,
        IOobject::AUTO_WRITE
    ),
    mesh
);

#include "createPhi.H"

label pRefCell = 0;
scalar pRefValue = 0.0;
setRefCell(p, pimple.dict(), pRefCell, pRefValue);
mesh.setFluxRequired(p.name());

```

```

singlePhaseTransportModel laminarTransport(U, phi);

autoPtr<incompressible::turbulenceModel> turbulence
(
    incompressible::turbulenceModel::New(U, phi, laminarTransport)
);

#include "createMRF.H"
#include "createFvOptions.H"

Info << "Reading field porosity\n" << endl;
volScalarField porosity
(
    IOobject
    (
        "porosity",
        runTime.timeName(),
        mesh,
        IOobject::MUST_READ,
        IOobject::AUTO_WRITE
    ),
    mesh
);

Info << "Reading field K\n" << endl;
volScalarField K
(
    IOobject
    (
        "K",
        runTime.timeName(),
        mesh,
        IOobject::MUST_READ,
        IOobject::AUTO_WRITE
    ),
    mesh

```

```

);

Info << "Creating averaged mass flux\n" << endl;
surfaceScalarField phiByPorosity
(
    IOobject
    (
        "phiByPorosity",
        runTime.timeName(),
        mesh,
        IOobject::READ_IF_PRESENT,
        IOobject::AUTO_WRITE
    ),
    phi*linearInterpolate(1./(porosity+SMALL))
);

dimensionedScalar nu ("nu", dimensionSet(0,2,-1,0,0,0,0), gMax(fvc::avera
ge(laminarTransport.nu() )()));
volScalarField darcyTerm ("darcyTerm", nu/K);

```

- ▼ **createFieldsBiofilm.H** : Inicializa las variables de la fase sólida y biológica:  $M$  (biomasa desnitrificante),  $M_{hid}$  (biomasa hidrolítica) y  $S_0$  (azufre sólido). Lee constantes físicas críticas como  $\rho_{S_0}$  (densidad azufre) y  $m_{max}$  (densidad máx. biofilm).

```

// === createFieldsBiofilm.H (Hito4-R2 - corregido) ===

// --- C (del solver original). Se puede desactivar con useC en controlDict.
Info << "Reading field C\n" << endl;

volScalarField C
(
    IOobject
    (
        "C",

```

```

        runTime.timeName(),
        mesh,
        IOobject::MUST_READ,
        IOobject::AUTO_WRITE
    ),
    mesh
);

// Monitores (opcionales)
dimensionedScalar ddtC("ddtC", dimensionSet(0,0,0,0,0,0,0), 0.);

// === BIOMASAS ===
// M del solver base se interpreta como biomasa desnitrificante X_desn
Info << "Reading field M (denitrifying biomass)\n" << endl;

volScalarField M
(
    IOobject
    (
        "M",
        runTime.timeName(),
        mesh,
        IOobject::MUST_READ,
        IOobject::AUTO_WRITE
    ),
    mesh
);

// Alias por claridad
volScalarField& Mdesn = M;

// Biomasa hidrolítica X_hid para S0 → Sb (nueva en Hito4)
Info << "Reading field Mhid (hydrolytic biomass)\n" << endl;

volScalarField Mhid
(

```

```

IOobject
(
    "Mhid",
    runTime.timeName(),
    mesh,
    IOobject::MUST_READ,    // requiere 0/Mhid
    IOobject::AUTO_WRITE
),
mesh
);

// Monitores de biomasa (opcionales)
dimensionedScalar ddtMdesn("ddtMdesn", dimensionSet(0,0,0,0,0,0,0),
0.);
dimensionedScalar ddtMhid ("ddtMhid", dimensionSet(0,0,0,0,0,0,0), 0.);

// Azufre elemental S0 (estado — sin advección)
Info << "Reading field S0 (elemental sulfur)\n" << endl;

volScalarField S0
(
    IOobject
    (
        "S0",
        runTime.timeName(),
        mesh,
        IOobject::MUST_READ,    // requiere 0/S0
        IOobject::AUTO_WRITE
    ),
    mesh
);

// --- Parámetros del micro-continuum del solver base
Info << "Reading transportProperties\n" << endl;

IOdictionary transportProperties

```

```

(
    IObject
    (
        "transportProperties",
        runTime.constant(),
        mesh,
        IObject::MUST_READ_IF_MODIFIED,
        IObject::NO_WRITE
    )
);

// dimensiones: kg | m | s | K | mol | A | cd
Info << "Reading parameter d1 \n" << endl;
dimensionedScalar d1 ("d1", dimensionSet(0, 2, -1, 0, 0, 0, 0), transportProp
erties);

Info << "Reading parameter mmax \n" << endl;
dimensionedScalar mmax ("mmax", dimensionSet(1, -3, 0, 0, 0, 0, 0), trans
portProperties);

// -----
// Campo adimensional de biomasa total normalizada:
// Mnorm = (M + Mhid)/mmax  (0 ~ sin biomasa, 1 ~ biomasa al máximo)
// -----
volScalarField Mnorm
(
    IObject
    (
        "Mnorm",
        runTime.timeName(),
        mesh,
        IObject::NO_READ,    // no se lee desde disco
        IObject::AUTO_WRITE // se escribe en cada time-step
    ),
    (M + Mhid)/mmax          // adimensional
);

```

```
Info << "Reading sulfur density rhoS0 \n" << endl;
```

```
dimensionedScalar rhoS0
```

```
(
```

```
    "rhoS0",
```

```
    dimensionSet(1,-3,0,0,0,0,0),
```

```
    transportProperties
```

```
);
```

```
Info << "Reading parameter c0 \n" << endl;
```

```
dimensionedScalar c0 ("c0", dimensionSet(1, -3, 0, 0, 0, 0, 0), transportPro  
perties);
```

```
Info << "Reading parameter mum \n" << endl;
```

```
dimensionedScalar mum ("mum", dimensionSet(0, 0, -1, 0, 0, 0, 0), transpo  
rtProperties);
```

```
Info << "Reading parameter Yxs \n" << endl;
```

```
dimensionedScalar Yxs ("Yxs", dimensionSet(0, 0, 0, 0, 0, 0, 0), transportPr  
operties);
```

```
Info << "Reading parameter ms \n" << endl;
```

```
dimensionedScalar ms ("ms", dimensionSet(0, 0, -1, 0, 0, 0, 0), transportPr  
operties);
```

```
Info << "Reading parameter Ks \n" << endl;
```

```
dimensionedScalar Ks ("Ks", dimensionSet(1, -3, 0, 0, 0, 0, 0), transportPro  
perties);
```

```
Info << "Reading parameter eps \n" << endl;
```

```
dimensionedScalar eps ("eps", dimensionSet(0, 0, 0, 0, 0, 0, 0), transportPr  
operties);
```

```
Info << "Reading parameter a \n" << endl;
```

```
dimensionedScalar a ("a", dimensionSet(0, 0, 0, 0, 0, 0, 0), transportProper
```

```

ties);

Info << "Reading parameter b \n" << endl;
dimensionedScalar b ("b", dimensionSet(0, 0, 0, 0, 0, 0, 0), transportProperties);

// Derivados (legado) para el modelo base
dimensionedScalar k1_legacy = mum/Yxs+ms;
dimensionedScalar k2_legacy = Ks;
dimensionedScalar k3_legacy = Yxs/mmax;
dimensionedScalar k4_legacy = ms*mmax;

dimensionedScalar K1_legacy = mmax*k1_legacy/c0;
dimensionedScalar K2_legacy = k2_legacy;
dimensionedScalar K3_legacy = mmax*k3_legacy*k1_legacy;
dimensionedScalar K4_legacy = k3_legacy*k4_legacy;

// --- Biofilm phase bookkeeping (sin cambios)
volScalarField biofilmPhase
(
    IOobject
    (
        "biofilmPhase",
        runTime.timeName(),
        mesh,
        IOobject::NO_READ,
        IOobject::AUTO_WRITE
    ),
    mesh,
    dimensionedScalar ("biofilmPhase", dimensionSet(0,0,0,0,0,0,0), 0.),
    "zeroGradient"
);

Info << "Reading biofilm permeability\n" << endl;
dimensionedScalar biofilmPermeability
(

```



```

transportProperties.lookupOrDefault<dimensionedScalar>
(
    "biofilmPermeability",
    dimensionedScalar("biofilmPermeability", dimLength*dimLength, 1e-1
2)
)
);

Info << "Reading biofilm porosity\n" << endl;
dimensionedScalar biofilmPorosity
(
    transportProperties.lookupOrDefault<dimensionedScalar>
    (
        "biofilmPorosity",
        dimensionedScalar("biofilmPorosity", dimless, 1)
    )
);

// --- NUEVO: parámetros hidráulicos para el lecho de S0 (pellets de azufr
e)

Info << "Reading sulfur permeability (S0 pellets)\n" << endl;
dimensionedScalar sulfurPermeability
(
    transportProperties.lookupOrDefault<dimensionedScalar>
    (
        "sulfurPermeability",
        dimensionedScalar("sulfurPermeability", dimLength*dimLength, 1e-14)
    )
);

Info << "Reading sulfur porosity (S0 pellets)\n" << endl;
dimensionedScalar sulfurPorosity
(
    transportProperties.lookupOrDefault<dimensionedScalar>

```

```

    (
        "sulfurPorosity",
        dimensionedScalar("sulfurPorosity", dimless, 0.1)
    )
);

Info << "Reading threshold S0phaseOn (pellet cutoff)\n" << endl;

dimensionedScalar S0phaseOn
(
    transportProperties.lookupOrDefault<dimensionedScalar>
    (
        "S0phaseOn",
        dimensionedScalar("S0phaseOn", dimMass/dimVolume, 1e-3) // valor
        razonable
    )
);

// --- HIDRÓLISIS S0 → Sb (Huiliñir 2020/2024) ---
//  $\mu_{hid} = k1 * S0 / (K1_{over\_aStar} + S0)$ 
// Nota: K1_over_aStar ya incluye el a* en el denominador del paper (K1/a*).

dimensionedScalar k1 ("k1", dimensionSet(0, 0, -1, 0, 0, 0, 0), transportP
roperties); // 1/s
dimensionedScalar aStar ("aStar", dimless, transportPr
operties); // -
/* Rendimiento de crecimiento hidrolítico (K0 del paper): adimensional (kg
X/kgS) */
dimensionedScalar K0 ("K0", dimless, transportPro
perties); // -
/* Decaimiento X_hid */
dimensionedScalar kd_hid ("kd_hid", dimensionSet(0,0,-1,0,0,0,0), transp
ortProperties);
/* Este factor suele ser 1.0 (todo S0 hidrolizado pasa a Sb) */

```

```

dimensionedScalar ySb_S0 ("ySb_S0", dimless, transportProperties);

/* Constante compuesta K1/a* expresada como concentración (kg/m3) */
dimensionedScalar K1_over_aStar
(
    transportProperties.lookupOrDefault<dimensionedScalar>
    (
        "K1_over_aStar",
        dimensionedScalar("K1_over_aStar", dimMass/dimVolume, 1e-6) // val
or por defecto prudente
    )
);

// --- Biomasa total (útil para porosidad/difusión)
volScalarField Mtot
(
    IOobject
    (
        "Mtot",
        runTime.timeName(),
        mesh,
        IOobject::NO_READ,
        IOobject::NO_WRITE
    ),
    Mdesn + Mhid
);

// === end createFieldsBiofilm.H (Hito4-R2) ===

```

▼ **createFieldsNOx.H** : Inicializa las especies químicas solubles y lee todas las constantes cinéticas del modelo de Huiliñir .

```

// === createFieldsNOx.H (Hito4-R2) ===

Info << "Reading field NO3\n" << endl;
volScalarField NO3
(
    IOobject("NO3", runTime.timeName(), mesh, IOobject::MUST_READ, IOo
bject::AUTO_WRITE),
    mesh
);

Info << "Reading field NO2\n" << endl;
volScalarField NO2
(
    IOobject("NO2", runTime.timeName(), mesh, IOobject::MUST_READ, IOo
bject::AUTO_WRITE),
    mesh
);

Info << "Reading field Csb (soluble sulfur donor)\n" << endl;
volScalarField Csb
(
    IOobject("Csb", runTime.timeName(), mesh, IOobject::MUST_READ, IOob
ject::AUTO_WRITE),
    mesh
);

Info << "Reading field SO4 (sulfate)\n" << endl;
volScalarField SO4
(
    IOobject("SO4", runTime.timeName(), mesh, IOobject::MUST_READ, IOo
bject::AUTO_WRITE),
    mesh
);

Info << "Reading field N2 (gaseous nitrogen tracker)\n" << endl;

```

```

volScalarField N2
(
    IOobject("N2", runTime.timeName(), mesh, IOobject::MUST_READ, IOobj
ect::AUTO_WRITE),
    mesh
);

// ----- Monitors (opcionales) -----
dimensionedScalar ddtNO3("ddtNO3", dimensionSet(0,0,0,0,0,0,0), 0.);
dimensionedScalar ddtNO2("ddtNO2", dimensionSet(0,0,0,0,0,0,0), 0.);
dimensionedScalar ddtSb ("ddtSb", dimensionSet(0,0,0,0,0,0,0), 0.);
dimensionedScalar ddtSO4("ddtSO4", dimensionSet(0,0,0,0,0,0,0), 0.);
dimensionedScalar ddtS0 ("ddtS0", dimensionSet(0,0,0,0,0,0,0), 0.);
dimensionedScalar ddtN2("ddtN2", dimensionSet(0,0,0,0,0,0,0), 0.);
// ----- Parámetros de transporte/kinética existentes (obligatorios) -----
---
Info << "Reading NOx transport/kinetics\n" << endl;
dimensionedScalar DNO3 ("DNO3", dimensionSet(0, 2, -1, 0, 0, 0, 0), trans
portProperties);
dimensionedScalar DNO2 ("DNO2", dimensionSet(0, 2, -1, 0, 0, 0, 0), trans
portProperties);
dimensionedScalar Dsb ("Dsb", dimensionSet(0, 2, -1, 0, 0, 0, 0), transpor
tProperties);
dimensionedScalar DSO4 ("DSO4", dimensionSet(0, 2, -1, 0, 0, 0, 0), trans
portProperties);
dimensionedScalar DN2 ("DN2", dimensionSet(0, 2, -1, 0, 0, 0, 0), transpor
tProperties);
dimensionedScalar kNO3 ("kNO3", dimensionSet(0, 0, -1, 0, 0, 0, 0), trans
portProperties); // 1/s
dimensionedScalar KNO3 ("KNO3", dimensionSet(1, -3, 0, 0, 0, 0, 0), trans
portProperties); // kg/m^3

dimensionedScalar kNO2 ("kNO2", dimensionSet(0, 0, -1, 0, 0, 0, 0), trans
portProperties); // 1/s
dimensionedScalar KNO2 ("KNO2", dimensionSet(1, -3, 0, 0, 0, 0, 0), trans
portProperties); // kg/m^3

```

```

dimensionedScalar KSb ("KSb", dimensionSet(1, -3, 0, 0, 0, 0, 0), transportProperties); // kg/m^3

// Yields/estequiometría (existentes)
dimensionedScalar YX_NO3("YX_NO3", dimless, transportProperties); // crecimiento X_desn por NO3
dimensionedScalar YX_NO2("YX_NO2", dimless, transportProperties); // crecimiento X_desn por NO2
dimensionedScalar ySO4_NO3("ySO4_NO3", dimless, transportProperties);
dimensionedScalar ySO4_NO2("ySO4_NO2", dimless, transportProperties);
dimensionedScalar ySb_NO3 ("ySb_NO3", dimless, transportProperties);
dimensionedScalar ySb_NO2 ("ySb_NO2", dimless, transportProperties);

// Decaimiento (existente: kd general). Creamos alias específico para X_desn.
dimensionedScalar kd ("kd", dimensionSet(0,0,-1,0,0,0,0), transportProperties);
dimensionedScalar kd_desn
(
    transportProperties.lookupOrDefault<dimensionedScalar>
    (
        "kd_desn", dimensionedScalar("kd_desn", kd.dimensions(), kd.value())
    )
);

// === end createFieldsNOx.H (Hito4-R2) ===

```

## Archivos de gestión de Fases y Geometría

▼ **updateBiofilmPhase.H** : Determina el estado de cada celda (Sólido S0, Biofilm o Líquido) basándose en umbrales físicos. Asigna la permeabilidad (K) y porosidad correctas, gestionando el efecto de "lecho fijo" que se consume.

- Si hay pellet de  $S_0$  → la celda es *soporte sólido*, no biofilm dominante.
- Si el  $S_0$  se agotó y hay biomasa → la celda es biofilm.
- Si no tiene ni  $S_0$  ni biomasa → fluido libre.

Si **hasSulfur** se apaga cuando S0 cae  $< S0_{phaseOn}$ ,  
y **hasBiofilm** se enciende cuando  $M_{norm} > SMALL$ ,  
entonces:

- el pellet se va gastando,
- aparece una zona donde el biofilm domina,
- el biofilmPhase avanza espacialmente.

#### Actualización de porosidad, K y darcyTerm

- Pellet de  $S_0$  → porosidad muy baja, K muy baja.
- Biofilm → porosidad intermedia, K baja.
- Fluido → porosidad 1,  $K=1$ , sin Darcy

```
// === updateBiofilmPhase.H — modo ORIGINAL usando Mnorm + manejo  
de S0 ===
```

```
// Atajos a los campos internos
```

```
scalarField& MField = M.primitiveFieldRef();
```

```
scalarField& MhidField = Mhid.primitiveFieldRef();
```

```
const scalarField& S0Field = S0.internalField(); // solo lectura
```

```
const scalarField& MnormField = Mnorm.internalField(); // solo lectura
```

```
scalarField& KField = K.primitiveFieldRef();
```

```
scalarField& epsField = porosity.primitiveFieldRef();
```

```

scalarField&    bfField  = biofilmPhase.primitiveFieldRef();
scalarField&    darcyField = darcyTerm.primitiveFieldRef();

// Umbral mínimo para considerar que hay pellet de S0
const scalar S0thVal = S0phaseOn.value(); // umbral físico
forAll (mesh.C(), celli)
{

    const scalar S0Cell  = S0Field[celli];
    const scalar MnormCell = MnormField[celli];
    const bool hasSulfur = (S0Cell > S0thVal);
    const bool hasBiofilm = (!hasSulfur && MnormCell > SMALL); // LÓGICA ORIGINAL

    if (hasSulfur)
    {
        // -----
        // 1) PELLET DE S0 — domina el sólido, NO hay biofilm
        // -----
        bfField[celli]  = 0.0;

        KField[celli]  = sulfurPermeability.value();
        epsField[celli] = sulfurPorosity.value();
        darcyField[celli] = nu.value()/KField[celli];

        // OPCIONAL: vaciar biomasa dentro del pellet
        // (No estaba en el original porque no existía S0)
        // MField[celli]  = 0.0;
        // MhidField[celli] = 0.0;
    }
    else if (hasBiofilm)
    {
        // -----
        // 2) BIOFILM — EXACTO COMO EL SOLVER ORIGINAL
        // -----
        bfField[celli]  = 1.0;
    }
}

```



```

    KField[celli] = biofilmPermeability.value();
    epsField[celli] = biofilmPorosity.value();
    darcyField[celli] = nu.value()/KField[celli];
}
else
{
    // -----
    // 3) FLUIDO LIBRE — IMPLEMENTACIÓN 100% ORIGINAL
    // -----
    bfField[celli] = 0.0;

    // Parámetros del fluido libre (igual que original)
    KField[celli] = 1.0;
    epsField[celli] = 1.0;
    darcyField[celli] = 0.0;

    // *** AQUÍ ESTÁ EL COMPORTAMIENTO ORIGINAL ***
    // En el solver original, M se pone a cero si M < SMALL.
    // Adaptación correcta: usar Mnorm en vez de M adimensional.
    if (MnormCell < SMALL)
    {
        MField[celli] = 0.0;
        MhidField[celli] = 0.0;
    }
}
}

// === Fin updateBiofilmPhase.H =====
=====

```

▼ **updateDiffusion.H** : Calcula coeficientes difusivos morfológicos

$$M_{\text{norm}} = \frac{M + M_{\text{hid}}}{m_{\text{max}}}$$

$$d_2 = \left( \frac{\varepsilon}{1 - M_{\text{norm}}} \right)^a (M_{\text{norm}})^b$$

```
// Actualizar campos Mtot y Mnorm
Mtot = M + Mhid;
Mnorm = Mtot/mmax;

// Proteger singularidades
Mnorm = min(max(Mnorm, SMALL), 1.0 - SMALL);

// d2 como en solver original, usando Mnorm adimensional
volScalarField d2 =
(
    Foam::pow(eps/(1 - Mnorm), a) *
    Foam::pow(Mnorm, b)
);

// Dimensiones físicas m^2/s
d2.dimensions().reset(dimensionSet(0,2,-1,0,0,0,0));

// Piso numérico
d2 = max(d2, dimensionedScalar("d2Floor", dimensionSet(0,2,-1,0,0,0,0), 1
e-20));

// En caras
surfaceScalarField d2f( fvc::interpolate(d2, "d2") );
```

▼ **updateInterfaceMS.H** : **Indice de dominancia de fase entre Mtot y S0.**

$$\chi_{MS} = \frac{\frac{M+M_{hid}}{m_{max}}}{\frac{M+M_{hid}}{m_{max}} + \frac{S_0}{\rho_{S0}}}$$

Representa la **interfaz biofilm–pellet**:

- $\chi = 0 \rightarrow$  la celda se comporta como  $S_0$  sólido
- $\chi = 1 \rightarrow$  la celda se comporta como biofilm
- $\chi = 0.5 \rightarrow$  frontera donde  $S_0$  se está agotando y el biofilm comienza a dominar

Sirve para:

- visualizar la interfaz biomasa– $S^0$
- seguir la invasión del pellet
- ver dónde la biomasa domina
- ver cómo la interfaz avanza con el tiempo

```
// === updateInterfaceMS.H: campo diagnóstico chi_MS =====
=====
// chi_MS indica quién domina localmente:
// - chi_MS ~ 0  → domina S0
// - chi_MS ~ 1  → domina biomasa (Mtot)
// - chi_MS ~ 0.5 → zona de interfaz S0 - biofilm

// Biomasa total (ya definido en createFieldsBiofilm.H):
// volScalarField Mtot = M + Mhid;

// Epsilon con mismas dimensiones que Mtot para evitar división por cero
const dimensionedScalar epsMS("epsMS", Mtot.dimensions(), SMALL);

// Factor físico alpha_MS = mmax / rhoS0 (adimensional)
dimensionedScalar alphaMS
(
    "alphaMS",
```

```

    dimless,
    mmax.value()/rhoS0.value()
);

// Denominador:  $M_{tot} + \alpha_{MS} * S_0 \approx$  masa de biofilm + masa equivalente de  $S_0$ 
volScalarField denomMS
(
    IOobject("denomMS", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::NO_WRITE),
    Mtot + alphaMS*S0 + epsMS
);

// Campo diagnóstico  $\chi_{MS}$  (adimensional, fracción volumétrica relativa)
volScalarField chi_MS
(
    IOobject
    (
        "chi_MS",
        runTime.timeName(),
        mesh,
        IOobject::NO_READ,
        IOobject::AUTO_WRITE
    ),
    Mtot/denomMS
);

// Acotar  $\chi_{MS}$  en [0, 1]
chi_MS = min
(
    max(chi_MS, dimensionedScalar("zero", dimless, 0.0)),
    dimensionedScalar("one", dimless, 1.0)
);

// (Opcional) log a consola
if (runTime.outputTime())

```

```

{
  Info<< "chi_MS[min,max]=[ " << gMin(chi_MS) << "," << gMax(chi_MS) <
  < "]" << nl;
}

// === fin updateInterfaceMS.H =====
=====

```

## Archivos de reacciones y transporte

▼ **NOxEqn.H (Solubles):** Resuelve el transporte y reacción de los sustratos solubles (NO<sub>3</sub>, NO<sub>2</sub>, C<sub>sb</sub>, SO<sub>4</sub>). Implementa el factor de limitación gamma, para asegurar el balance de masa del donador de electrones.

Cinéticas (Monod) y tasas

(1.1)–(1.2) factores de saturación

$$f_{Sb} = \frac{c_{Sb}}{K_{S,1} + c_{Sb}} \quad f_{NO_3} = \frac{c_{NO_3}}{K_{S,NO_3} + c_{NO_3}} \quad f_{NO_2} = \frac{c_{NO_2}}{K_{S,NO_2} + c_{NO_2}}$$

(1.1)–(1.2) tasas específicas autótrofas

$$\mu_{desn,2} = \mu_{max,2} f_{Sb} f_{NO_2} \quad \mu_{desn,1} = \mu_{max,1} f_{Sb} f_{NO_3}$$

$$r_{NO_3} = \frac{\mu_{desn,1} X_{desn}}{Y_{2,3}} \quad r_{NO_2} = \frac{\mu_{desn,2} X_{desn}}{Y_{2,4}}$$

y es un limitador estequiométrico para el donador de electrones *Sb*. Su función es evitar que, dentro de un paso de tiempo, el consumo requerido de *Sb* por las

rutas  $\text{NO}_3/\text{NO}_2$  supere lo que hay disponible más lo que se genera por hidrólisis  
 $S0 \rightarrow Sb$

$$\gamma = \min \left( 1, \frac{c_{Sb} + y_{Sb/S0} R_{\text{hyd}} \Delta t}{(y_{Sb,NO_3} r_{NO_3}^{\text{raw}} + y_{Sb,NO_2} r_{NO_2}^{\text{raw}}) \Delta t + \varepsilon} \right)$$

$$r_{NO_3} = \gamma r_{NO_3}^{\text{raw}}, \quad r_{NO_2} = \gamma r_{NO_2}^{\text{raw}}$$

$\text{NO}_3^-$  (paper 1.9)

$$\frac{\partial(\varepsilon c_{NO_3})}{\partial t} - \nabla \cdot (\varepsilon D_{NO_3} \nabla c_{NO_3}) = -r_{NO_3}$$

**Nitrito,  $\text{NO}_2^-$**

$$\frac{\partial(\varepsilon c_{NO_2})}{\partial t} - \nabla \cdot (\varepsilon D_{NO_2} \nabla c_{NO_2}) = r_{NO_3} - r_{NO_2}$$

$\text{N}_2$ :

$$\frac{\partial(\varepsilon c_{N_2})}{\partial t} - \nabla \cdot (\varepsilon D_{N_2} \nabla c_{N_2}) = r_{NO_2}$$

Sulfato,  $\text{SO}_4^{2-}$

$$\frac{\partial(\varepsilon c_{SO_4})}{\partial t} - \nabla \cdot (\varepsilon D_{SO_4} \nabla c_{SO_4}) = y_{SO_4,NO_3} r_{NO_3} + y_{SO_4,NO_2} r_{NO_2}$$

Donador soluble, Sb (Csb)

$$\frac{\partial(\varepsilon c_{Sb})}{\partial t} - \nabla \cdot (\varepsilon D_{Sb} \nabla c_{Sb}) = y_{Sb/S0} R_{\text{hyd}} - (y_{Sb,NO_3} r_{NO_3} + y_{Sb,NO_2} r_{NO_2})$$

$S0$

$$\frac{\partial S^0}{\partial t} = -R_{\text{hyd}}$$

$$R_{\text{hyd}} = \mu_{\text{hid}} M_{\text{hid}}$$

```
// ==== NOxEqn.H — Hito4-R7c: explícito + limitación por donador + integrales ====
```

```
// Pequeños epsilons y constantes con dimensiones correctas
const dimensionedScalar epsNO3("epsNO3", NO3.dimensions(), SMALL);
const dimensionedScalar epsNO2("epsNO2", NO2.dimensions(), SMALL);
const dimensionedScalar epsSb ("epsSb", Csb.dimensions(), SMALL);
const dimensionedScalar epsS ("epsS", Csb.dimensions(), SMALL);
const dimensionedScalar one ("one", dimless, 1.0);
```

```
// Δt con dimensiones de tiempo (s)
const dimensionedScalar dtDim("dt", dimTime, runTime.deltaTValue());
```

```
// Ceros con dimensión adecuada para recorte de positividad
const dimensionedScalar ZERO_NO3("ZERO_NO3", NO3.dimensions(), 0);
const dimensionedScalar ZERO_NO2("ZERO_NO2", NO2.dimensions(), 0);
const dimensionedScalar ZERO_S ("ZERO_S", Csb.dimensions(), 0);
const dimensionedScalar ZERO_SO4("ZERO_SO4", SO4.dimensions(), 0);
const dimensionedScalar ZERO_S0 ("ZERO_S0", S0.dimensions(), 0);
const dimensionedScalar ZERO_N2 ("ZERO_N2", N2.dimensions(), 0);
```

```
// Positividad
```

```
volScalarField NO3pos = max(NO3, ZERO_NO3);
volScalarField NO2pos = max(NO2, ZERO_NO2);
volScalarField SbPos = max(Csb, ZERO_S);
```

```
// Factores de Monod (usar Csb, no C)
```

```
volScalarField fSb = SbPos / (KSb + SbPos + epsSb);
volScalarField fNO3 = NO3pos / (KNO3 + NO3pos + epsNO3);
volScalarField fNO2 = NO2pos / (KNO2 + NO2pos + epsNO2);
```

```
// Tasas "crudas" explícitas (kg N / m3 / s), crecen con Mdesn (=M)
```

```

volScalarField rNO3_raw
(
    IOobject("rNO3_raw", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::NO_WRITE),
    (kNO3 * fSb * fNO3 * Mdesn)/YX_NO3
);
volScalarField rNO2_raw
(
    IOobject("rNO2_raw", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::NO_WRITE),
    (kNO2 * fSb * fNO2 * Mdesn)/YX_NO2
);

// ---- Limitación por donador (base S) ----
// Demanda de Sb del paso (kg S/m3) y Oferta del paso (kg S/m3)
volScalarField demandSb =
    (ySb_NO3 * rNO3_raw + ySb_NO2 * rNO2_raw) * dtDim;

volScalarField supplySb =
    SbPos + ySb_S0 * R_hyd * dtDim;

// Factor limitante gamma (adimensional)
volScalarField gamma
(
    IOobject("gamma", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::AUTO_WRITE),
    min(one, supplySb / (demandSb + epsS))
);

// Tasas efectivas limitadas por donador
volScalarField rNO3
(
    IOobject("rNO3", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::NO_WRITE),
    gamma * rNO3_raw
);

```



```

volScalarField rNO2
(
    IOobject("rNO2", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::NO_WRITE),
    gamma * rNO2_raw
);

// ===== Integrales acumuladas (kg) para diagnóstico =====
// Nota: acumulamos con integrales de tasa volumétrica * dt.
// Se guardan como estáticos para persistir durante la corrida.
static scalar A_NO3kg = 0.0; // consumo integrado por vía NO3 (kg N)
static scalar A_NO2kg = 0.0; // consumo integrado por vía NO2 (kg N)
static scalar A_Xkg = 0.0; // producción integrada de X_desn (kg X)
static scalar A_N2kg = 0.0; // producción integrada de N2 (kg N)
// incrementos del paso (kg)
const scalar dNO3_step =
    fvc::domainIntegrate(rNO3).value() * runTime.deltaTValue();

const scalar dNO2_step =
    fvc::domainIntegrate(rNO2).value() * runTime.deltaTValue();

const scalar dX_step =
    fvc::domainIntegrate(YX_NO3*rNO3 + YX_NO2*rNO2).value()
    * runTime.deltaTValue();

const scalar dN2_step =
    fvc::domainIntegrate(rNO2).value() * runTime.deltaTValue();
// acumular
A_NO3kg += dNO3_step;
A_NO2kg += dNO2_step;
A_Xkg += dX_step;
A_N2kg += dN2_step;
// --- Diagnóstico en cada writeInterval: gamma, balances y acumulados ---
-
if (runTime.outputTime())
{

```

```

// Paso: oferta/demanda de Sb en kg (ya están en kg/m3 → integrar a kg)
const dimensionedScalar demKg = fvc::domainIntegrate(demandSb);
const dimensionedScalar supKg = fvc::domainIntegrate(supplySb);

Info<< "gamma[min,max]=[ " << gMin(gamma) << ", " << gMax(gamma)
<< "]" << nl;
<< "Sb_demand_step=" << demKg.value() << " kgS "
<< "Sb_supply_step=" << supKg.value() << " kgS" << nl;

// Totales de dominio para balances (en kg)
const dimensionedScalar S0_sum = fvc::domainIntegrate(S0);
const dimensionedScalar Sb_sum = fvc::domainIntegrate(Csb);
const dimensionedScalar SO4_sum = fvc::domainIntegrate(SO4);
const dimensionedScalar N3_sum = fvc::domainIntegrate(NO3);
const dimensionedScalar N2_sum = fvc::domainIntegrate(NO2);
const dimensionedScalar M_sum = fvc::domainIntegrate(M);
const dimensionedScalar Mh_sum = fvc::domainIntegrate(Mhid);
const dimensionedScalar N2gas_sum = fvc::domainIntegrate(N2);
Info<< "TOT (kg): S0=" << S0_sum.value()
<< ", Sb=" << Sb_sum.value()
<< ", SO4=" << SO4_sum.value()
<< ", NO3=" << N3_sum.value()
<< ", NO2=" << N2_sum.value()
<< ", N2=" << N2gas_sum.value()
<< ", M=" << M_sum.value()
<< ", Mhid=" << Mh_sum.value()
<< nl;

// Línea de integrales acumuladas para comparar con AWK
Info<< "INT (kg): NO3=" << A_NO3kg
<< ", NO2=" << A_NO2kg
<< ", N2=" << A_N2kg
<< ", Xdesn=" << A_Xkg
<< nl;
}

```

```

// ===== NO3 (consumo) =====
// ===== NO3 =====
fvScalarMatrix NO3Eqn
(
    fvm::ddt(porosity, NO3)
    - fvm::laplacian(DNO3*porosity, NO3)
    == - rNO3
);

// HITO4: advección sólo en fase fluido (se apaga en biofilm con 'porosity')
if (advectionSolves)
{
    NO3Eqn += porosity * fvm::div(phiByPorosity, NO3);
}

NO3Eqn.solve();
NO3.correctBoundaryConditions();
NO3 = Foam::max(NO3, ZERO_NO3);

// ===== NO2 (prod - cons) =====
fvScalarMatrix NO2Eqn
(
    fvm::ddt(porosity, NO2)
    - fvm::laplacian(DNO2*porosity, NO2)
    == rNO3 - rNO2
);

// HITO4: advección solo en fase fluido (se apaga en biofilm con 'porosity')
if (advectionSolves)
{
    NO2Eqn += porosity * fvm::div(phiByPorosity, NO2);
}

NO2Eqn.solve();
NO2.correctBoundaryConditions();
NO2 = max(NO2, ZERO_NO2);

```

```

// ===== Csb (Sb) =====
// + hidrólisis – consumos por ambas vías (USANDO rNOx EFECTIVOS)
fvScalarMatrix CsbEqn
(
    fvm::ddt(porosity, Csb)
    - fvm::laplacian(Dsb*porosity, Csb)
    == ySb_S0 * R_hyd
    - (ySb_NO3 * rNO3 + ySb_NO2 * rNO2)
);
// HITO4: advección solo en fase fluido (se apaga dentro de biofilm con 'po
rosity')
if (advectionSolutions)
{
    CsbEqn += porosity * fvm::div(phiByPorosity, Csb);
}
CsbEqn.solve();
Csb.correctBoundaryConditions();
Csb = max(Csb, ZERO_S);

// ===== SO4 (base S) =====
fvScalarMatrix SO4Eqn
(
    fvm::ddt(porosity, SO4)
    - fvm::laplacian(DSO4*porosity, SO4)
    == ySO4_NO3 * rNO3 + ySO4_NO2 * rNO2
);
// HITO4: advección solo en fase fluido (se apaga dentro de biofilm con 'po
rosity')
if (advectionSolutions)
{
    SO4Eqn += porosity * fvm::div(phiByPorosity, SO4);
}

SO4Eqn.solve();
SO4.correctBoundaryConditions();
SO4 = max(SO4, ZERO_SO4);

```

```

// ===== N2 (nitrógeno gaseoso disuelto) =====
// =====
fvScalarMatrix N2Eqn
(
    fvm::ddt(porosity, N2)
    - fvm::laplacian(DN2*porosity, N2)
    == rNO2    // producción de N en forma N2 por consumo de NO2
);

// HITO4: advección solo en fase fluido (se apaga dentro de biofilm con 'po
rosity')
if (advectionSolut)
{
    N2Eqn += porosity * fvm::div(phiByPorosity, N2);
}

N2Eqn.solve();
N2.correctBoundaryConditions();
N2 = max(N2, ZERO_N2);

// ===== S0 (sólido inmóvil) =====
fvScalarMatrix S0Eqn
(
    fvm::ddt(S0) == - R_hyd
);
S0Eqn.solve();
S0.correctBoundaryConditions();
S0 = max(S0, ZERO_S0);

// (Opcional) monitores de derivadas normalizadas
ddtNO3 = gMax(mag(fvc::ddt(NO3)))/gMax(NO3)+SMALL;
ddtNO2 = gMax(mag(fvc::ddt(NO2)))/gMax(NO2)+SMALL;
ddtSb = gMax(mag(fvc::ddt(Csb)))/gMax(Csb)+SMALL;

```

```

ddtSO4 = gMax(mag(fvc::ddt(SO4)())/(gMax(SO4)+SMALL);
ddtSO  = gMax(mag(fvc::ddt(SO)())/(gMax(SO) +SMALL);
ddtN2  = gMax(mag(fvc::ddt(N2)())/(gMax(N2) +SMALL);
// ---- Monitor de la biomasa total, análogo al del solver original ----
// Actualizamos Mtot para este paso
Mtot = M + Mhid;

// Monitor al estilo del solver original pero con Mtot
dimensionedScalar ddtMtot
(
    "ddtMtot", dimless,
    gMax(mag(fvc::ddt(Mtot)())/(gMax(Mtot) + SMALL)
);

if (runTime.outputTime())
{
    Info<< "Monitores biofilm: ddtMtot=" << ddtMtot.value()
        << ", ddtMdesn=" << ddtMdesn.value()
        << ", ddtMhid=" << ddtMhid.value() << nl;
}
// ==== fin NOxEqn.H =====
=====

```

## ▼ Biomosas

Se incorporaron dos poblaciones:

- **M** = biomasa desnitrificante
- **Mhid** = biomasa hidrolítica

Cada una con **dimensiones físicas (kg/m³)**, lo cual es un cambio importante respecto al solver original, donde M era **adimensional**.

$$M_{\text{tot}} = M + M_{\text{hid}}$$

## Normalización de biomasa: creación de $M_{\text{norm}}$

$$M_{\text{norm}} = \frac{M_{\text{tot}}}{m_{\text{max}}}$$

donde  $m_{\text{max}}$  es la densidad máxima de biofilm ( $\text{kg/m}^3$ ).

Esto deja:

- **$M_{\text{norm}}$  adimensional,**
- **en el rango 0–1,**
- con significado idéntico a la  **$M$  del solver original.**

**MEqn.H (Crecimiento Biológico):** Resuelve la ecuación de la biomasa desnitrificante ( $M$ ). Gestiona su crecimiento (basado en el consumo de  $\text{NO}_x$ ) y su expansión espacial mediante el término Laplaciano ( $d_2$ ).

$$R_{g,M} = Y_{X/\text{NO}_3} r_{\text{NO}_3} + Y_{X/\text{NO}_2} r_{\text{NO}_2}$$

$$\frac{\partial X_{\text{desn}}}{\partial t} - \nabla \cdot (d_2(X_{\text{tot}}) \nabla X_{\text{desn}}) = R_{g,M} - k_{d,\text{desn}} X_{\text{desn}}$$

MEqn.H:

```
// crecimiento consistente con NOxEqn (tasas EFECTIVAS)
volScalarField Rg_M
(
    IOobject("Rg_M", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::AUTO_WRITE),
    YX_NO3 * rNO3 + YX_NO2 * rNO2
);

fvScalarMatrix MEqn
(
    fvm::ddt(M)
```

```

- fvm::laplacian(d2, M, "laplacian(d2,M)")
+ fvm::Sp(kd_desn, M)
== Rg_M
);
MEqn.relax();
MEqn.solve();

// --- DEBUG R9 ---
Info<< "MEqn R9 activo. "
    << "Rg_M[min,max]=[ " << gMin(Rg_M) << ", " << gMax(Rg_M) << " ] "
    << "M[min,max]=[ " << gMin(M) << ", " << gMax(M) << " ]\n";

```

**MHidEqn.H (Hidrólisis):** Calcula la tasa de hidrólisis superficial ( $R_{hyd}$ ). Resuelve la ecuación de consumo del azufre sólido:  $dS_0/dt = -R_{hyd}$ .

$$\mu_{hid} = \frac{k_1 S^0}{K_1/a^* + S^0}$$

$$\frac{\partial M_{hid}}{\partial t} - \nabla \cdot (d_2 \nabla M_{hid}) = K_0 \mu_{hid} M_{hid} - k_{d,hid} M_{hid}$$

```

// === MHidEqn.H: hidrólisis S0→Sb + difusión de biomasa hidrolítica ===
===
// Evitar divisiones por cero (mismas dimensiones que S0)
const dimensionedScalar tinyS0("tinyS0", S0.dimensions(), SMALL);

// μ_hidr = k1 * S0 / (K1_over_aStar + S0)
volScalarField mu_hid
(
    IOobject("mu_hid", runTime.timeName(), mesh, IOobject::NO_READ, IOob
ject::NO_WRITE),
    k1 * S0 / (K1_over_aStar + S0 + tinyS0) // [1/s]
);

```



```

// Tasa de hidrólisis volumétrica (kg S0/m3/s), se mantiene igual
volScalarField R_hyd
(
    IOobject("R_hyd", runTime.timeName(), mesh, IOobject::NO_READ, IOobject::AUTO_WRITE),
    mu_hid * Mhid
);

// PDE con difusión:
//  $\frac{dM_{hid}}{dt} - \nabla \cdot (D \nabla M_{hid}) = (K_0 \mu_{hid} - k_d) M_{hid}$ 
fvScalarMatrix MHidEqn
(
    fvm::ddt(Mhid)
    - fvm::laplacian(D, Mhid)
    ==
    (K0*mu_hid - kd_hid) * Mhid
);

MHidEqn.solve();
// =====
=====

```

## Hidrodinámica y Control Numérico

▼ **UEqn.H (Momento):** Resuelve las ecuaciones de Navier-Stokes para la velocidad del fluido. Incluye el término de Darcy que frena el flujo en las zonas marcadas como S0 o Biofilm.

```

// Solve the Momentum equation

```

```

MRF.correctBoundaryVelocity(U);

tmp<fvVectorMatrix> tUEqn
(
    (1./porosity+SMALL) * fvm::ddt(U)
    + fvm::div(phiByPorosity, U)
    - fvm::laplacian(laminarTransport.nu()/(porosity+SMALL), U)
    + fvm::Sp(darcyTerm, U)
    ==
    fvOptions(U)

);

fvVectorMatrix& UEqn = tUEqn.ref();

UEqn.relax();

fvOptions.constrain(UEqn);

if (pimple.momentumPredictor())
{
    solve(UEqn == -fvc::grad(p));

    fvOptions.correct(U);
}

```

▼ **pEqn.H (Presión):** Resuelve la ecuación de presión para garantizar la continuidad de masa (conservación) en el fluido incompresible, ajustando el flujo másico  $\phi$  en el medio poroso.

```

volScalarField rAU(1.0/UEqn.A());
volVectorField HbyA(constrainHbyA(rAU*UEqn.H(), U, p));
surfaceScalarField phiHbyA("phiHbyA", fvc::flux(HbyA));

if (pimple.ddtCorr())
{

```

```

    phiHbyA += MRF.zeroFilter(fvc::interpolate(rAU)*fvc::ddtCorr(U, phi, U
f));
}
else
{
    phiHbyA += MRF.zeroFilter(fvc::interpolate(rAU));
}

MRF.makeRelative(phiHbyA);

if (p.needReference())
{
    fvc::makeRelative(phiHbyA, U);
    adjustPhi(phiHbyA, U, p);
    fvc::makeAbsolute(phiHbyA, U);
}

tmp<volScalarField> rAtU(rAU);

if (pimple.consistent())
{
    rAtU = 1.0/max(1.0/rAU - UEqn.H1(), 0.1/rAU);
    phiHbyA +=
        fvc::interpolate(rAtU() - rAU)*fvc::snGrad(p)*mesh.magSf();
    HbyA -= (rAU - rAtU())*fvc::grad(p);
}

if (pimple.nCorrPISO() <= 1)
{
    tUEqn.clear();
}

// Update the pressure BCs to ensure flux consistency
constrainPressure(p, U, phiHbyA, rAtU(), MRF);

// Non-orthogonal pressure corrector loop

```

```

while (pimple.correctNonOrthogonal())
{
    fvScalarMatrix pEqn
    (
        fvm::laplacian(rAtU(), p) == fvc::div(phiHbyA)
    );

    pEqn.setReference(pRefCell, pRefValue);

    pEqn.solve(mesh.solver(p.select(pimple.finalInnerIter())));

    if (pimple.finalNonOrthogonalIter())
    {
        phi = phiHbyA - pEqn.flux();
    }
}

#include "continuityErrs.H"

// Explicitly relax pressure for momentum corrector
p.relax();

U = HbyA - rAtU*fvc::grad(p);
U.correctBoundaryConditions();
fvOptions.correct(U);

// Correct Uf if the mesh is moving
fvc::correctUf(Uf, U, phi);

// Make the fluxes relative to the mesh motion
fvc::makeRelative(phi, U);

phiByPorosity = phi*linearInterpolate(1./(porosity+SMALL));

```

▼ **setDeltaT.H** y **readTimeControls.H** : Controlan el paso de tiempo ( $\Delta t$ ) de forma adaptativa. Si la reacción química es muy rápida, reducen el paso de tiempo para mantener la estabilidad y precisión (basado en Courant y cambios en M,NO3,NO2).

```
/*-----
*\
```

```
]3 | () /= | | _ |V| /= () ^ |V|
```

Copyright 2023 Toulouse INP  
Romain Guibert

#### License

This file is part of biofilmFoam, an extension of OpenFOAM.

OpenFOAM is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

OpenFOAM is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or

FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with OpenFOAM. If not, see <<http://www.gnu.org/licenses/>>.

Global

setDeltaT

```
\*-----  
*/  
  
if (adjustTimeStep)  
{  
    // Límite por Courant (requiere haber incluido CourantNo.H antes)  
    scalar maxDeltaTFact = maxCo/(CoNum + SMALL);  
    scalar dtGrowthSlope = runTime.controlDict().lookupOrDefault<scalar>  
("dtGrowthSlope", 0.5);  
    scalar dtGrowthCap = runTime.controlDict().lookupOrDefault<scalar>  
("dtGrowthCap", 1.8);  
    scalar deltaTFact = min( min(maxDeltaTFact, 1.0 + dtGrowthSlope*maxD  
eltaTFact), dtGrowthCap );  
  
    // Sugerencias de los gestores por variable  
    scalar dtForM = dtManagerM.computeTimestep();  
    scalar dtForNO3 = dtManagerNO3.computeTimestep();  
    scalar dtForNO2 = dtManagerNO2.computeTimestep();  
  
    // Si C está desactivado, no lo consideres (usa GREAT para que no limit  
e)  
    scalar dtForC = useC ? dtManagerC.computeTimestep() : GREAT;  
  
    // Más restrictivo entre todos los activos  
    scalar dtFromManagers = min(min(dtForM, dtForNO3), min(dtForNO2, dt  
ForC));  
  
    // Aplica también el límite por Co y maxDeltaT  
    runTime.setDeltaT  
    (  
        min  
        (  
            dtFromManagers,  
            min(deltaTFact*runTime.deltaTValue(), maxDeltaT)
```

```

    )
);

Info<< "Δt managers: "
    << "M=" << dtForM
    << ", NO3=" << dtForNO3
    << ", NO2=" << dtForNO2;

if (useC) Info << ", C=" << dtForC; else Info << ", C=off";

Info<< " | elegido=" << runTime.deltaTValue()
    << " (factor Co=" << deltaTFact << ")"
    << nl << endl;
}
// *****
**** //

```

```

/*-----
* \

```

$$|3| \vee () \neq | \_ |V| \neq () \wedge |V|$$

Copyright 2023 Toulouse INP  
 Romain Guibert

## License

This file is part of biofilmFoam, an extension of OpenFOAM.

OpenFOAM is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

OpenFOAM is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or

FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with OpenFOAM. If not, see <http://www.gnu.org/licenses/>.

Global

readTimeControls

```
\*-----  
*/
```

```
//const bool adjustTimeStep =  
//runTime.controlDict().getOrDefault("adjustTimeStep", false);
```

```
//scalar maxDeltaT =  
//runTime.controlDict().getOrDefault<scalar>("maxDeltaT", GREAT);
```

```
//scalar maxCo =  
//runTime.controlDict().getOrDefault<scalar>("maxCo", 1);
```

```
const scalar truncationError_global =  
runTime.controlDict().lookupOrDefault<scalar>("truncationError",0.01);
```

```
const scalar truncationError_C =  
runTime.controlDict().lookupOrDefault<scalar>("truncationError_C",truncati  
onError_global);
```

```
const scalar truncationError_M =  
runTime.controlDict().lookupOrDefault<scalar>("truncationError_M",truncat  
ionError_global);
```



```
// NUEVO: tolerancias específicas para NO3 y NO2 (opcionalmente en controlDict)
const scalar truncationError_NO3 =
    runTime.controlDict().lookupOrDefault<scalar>("truncationError_NO3", truncationError_global);

const scalar truncationError_NO2 =
    runTime.controlDict().lookupOrDefault<scalar>("truncationError_NO2", truncationError_global);
timestepManager dtManagerC(runTime, C, truncationError_C);
timestepManager dtManagerM(runTime, M, truncationError_M);
// NUEVO: añadir control para NOx (transitorios rápidos, picos de nitrito)
timestepManager dtManagerNO3(runTime, NO3, truncationError_NO3);
timestepManager dtManagerNO2(runTime, NO2, truncationError_NO2);
// *****
**** //
```

▼ **correctPhi.H** : Ajusta el flujo inicial para que respete la física e informa si la masa se está conservando bien antes de empezar a calcular química y biomasa

```
CorrectPhi
(
    U,
    phi,
    p,
    dimensionedScalar("rAUf", dimTime, 1),
    geometricZeroField(),
    pimple
);

#include "continuityErrs.H"
```

▼ **pimple.H** : Controla el bucle iterativo PIMPLE (corrector presión-momento) dentro de cada paso de tiempo.

```
// --- Pressure-velocity PIMPLE corrector loop
while (pimple.loop())
{
    if (pimple.firstIter() || moveMeshOuterCorrectors)
    {
        // Do any mesh changes
        mesh.controlledUpdate();

        if (mesh.changing())
        {
            MRF.update();

            if (correctPhi)
            {
                // Calculate absolute flux
                // from the mapped surface velocity
                phi = mesh.Sf() & Uf();

                #include "correctPhi.H"

                // Make the flux relative to the mesh motion
                fvc::makeRelative(phi, U);
            }

            if (checkMeshCourantNo)
            {
                #include "meshCourantNo.H"
            }
        }
    }

    // #include "updateFields.H"
```

```

#include "UEqn.H"

// --- Pressure corrector loop
while (pimple.correct())
{
    #include "pEqn.H"
}

if (pimple.turbCorr())
{
    laminarTransport.correct();
    turbulence->correct();
}
}

```

▼ **CEqn.H** (Legado): Ecuación heredada del solver original , se mantiene por compatibilidad pero puede desactivarse (useC false).

```

fvScalarMatrix CEqn
(
    fvm::ddt(porosity,C)
    + porosity * fvm::div(phiByPorosity, C)
    - fvm::laplacian(d1*porosity, C)
    ==
    - fvm::Sp(dimensionedScalar("zeroC", dimensionSet(0,0,-1,0,0,0,0), 0),
    C)
);

solve(CEqn);

```

```
ddtC = gMax(mag(fvc::ddt(C))())/ (gMax(C)+SMALL);
```